

# Performance Pass — Fast Navigation & Caching

Helix Studio · shipped 2026-06-12, alongside the Intent Ledger release

## The diagnosis

- 1. **No `loading.tsx` anywhere** — page transitions showed a frozen screen until all server queries finished.
- 2. **`auth()` ran un-deduplicated** — the layout and the page each decoded the session on every navigation.
- 3. **Sequential DB queries** in the shared-workspace editor path (3 chained round-trips).
- 4. **Zero client-side caching** — revisiting any screen refetched everything from scratch.
- 5. **IMPORT-mode workspaces refetched the entire GitHub repo tree** on every file list.
- 6. **`getGitAuth` paid 2 DB round-trips** on nearly every workspace API call.
- 7. Command palette / avatar dropdown navigated with `router.push` — no prefetch.

## The fixes

| Area                    | Change                                                                                                                  | File(s)                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Instant transitions     | Route-group skeleton + editor-shaped skeleton; the shell stays mounted and the content area paints immediately          | <code>src/app/(app)/loading.tsx</code> ,<br><code>src/app/(app)/editor/[id]/loading.tsx</code>          |
| Session dedupe          | <code>auth()</code> wrapped in React <code>cache()</code> — one decode per request                                      | <code>src/lib/auth.ts</code>                                                                            |
| Parallel queries        | Membership + owner lookups in one round-trip                                                                            | <code>editor/[id]/page.tsx</code>                                                                       |
| Client cache (SWR-lite) | Module-level cache: revisits paint last-known data instantly, fetch refreshes in background                             | <code>src/lib/client-cache.ts</code> + workspace tree, chat history, Space activity / gradebook / board |
| Repo tree cache         | 60 s TTL per <code>provider:repo@branch</code> — kills the GitHub round-trip on repeat file lists                       | <code>src/lib/workspace.ts</code>                                                                       |
| Git auth cache          | 60 s TTL, positive results only, invalidated on settings save                                                           | <code>src/lib/git/index.ts</code> , preferences route                                                   |
| Prefetch                | Palette warms <code>/</code> , <code>/editor</code> , <code>/settings</code> on open; rail warms <code>/settings</code> | <code>command-palette.tsx</code> , <code>rail.tsx</code>                                                |
| Fonts                   | <code>display: "swap"</code> — text paints with fallbacks instead of blocking                                           | <code>src/app/layout.tsx</code>                                                                         |

## Live preview fix (Run app hang)

A Vite app could sit on the boot screen indefinitely. Root causes found and fixed in `src/lib/runner/local.ts` :

- 1. **ANSI color codes broke port discovery** — Vite bolds the port mid-URL ( `http://127.0.0.1: + ESC[1m + 5173` ), so the announcement regex never matched. Logs are now ANSI-stripped before parsing (and read cleanly in the UI).
- 2. **No boot watchdog** — any wedge spun the spinner forever. A run that isn't reachable within **4 minutes** now flips to `error` with an actionable log line and kills the child process.
- 3. Next 16 dev runs route handlers in recyclable worker processes; a recycled worker loses the in-memory run registry. That case now degrades to a clean "stopped" state instead of a phantom spinner.

Verified: `node scripts/repro-vite-run.mjs` — a Vite + React app goes from Run click to reachable preview in ~5–10 s.

## What's deliberately NOT cached

---

- Rate limiting still hits Redis per request (correctness > latency for abuse control).
- File *content* reads stay fresh (the editor's source of truth).
- Negative git-auth results are never cached — connecting a host works immediately.